

MIPS32 Instruction Set

A Deep, Intuitive Study Guide

Course: ICT 1022 2.0 – Computer System Architecture (2 credits)

Source lecture: Dr. Surani Tissera, Dept. of Computer Science, University of Sri Jayewardenepura

How to use this guide: Every topic follows the same three-step rhythm – *What it is* → *Why it exists* → *How to recognise/use it*. Green boxes explain the underlying engineering reason; blue boxes give an everyday analogy; yellow boxes are exam-style tips; orange boxes are memory aids. Read the 'Why' boxes even if you skim everything else – understanding *why* a feature exists makes the mnemonics memorise themselves.

Learning objectives covered:

- Explain the philosophy of a load-store architecture and why memory alignment matters.
- Classify every MIPS32 instruction into its functional group and encoding format.
- Reconstruct the R-type, I-type, and J-type bit layouts from first principles, not memorisation.
- Translate short C statements into correct MIPS32 assembly, choosing sensible registers.
- Explain each of the five addressing modes with a worked example.

1. The Big Picture: Why Classify Instructions at All?

Before memorising anything, ask: *why does a CPU designer bother sorting instructions into groups?* The answer is that the classification mirrors how the hardware itself is built. A CPU is not one monolithic black box – it is a collection of specialised units (an ALU, a memory-access unit, a branch unit, etc.), each wired to handle one family of instructions efficiently. Learning the classification is really learning the **floor plan of the processor**.

Why it matters: Every instruction is exactly 32 bits wide, no exceptions. This uniform width is what allows the CPU to fetch one instruction per cycle without first checking 'how long is this instruction?' (unlike, say, x86, where instructions vary from 1 to 15 bytes and the decoder must work harder). Fixed width is one of the defining features of a RISC (Reduced Instruction Set Computer) design.

The five functional groups, and the one question each answers:

Group	The question it answers
Load and Store	How do I move data between memory and registers?
Arithmetic and Logical	How do I compute a new value from existing register values?
Jump and Branch	How do I change which instruction runs next?
Miscellaneous	How do I handle exceptions, hints, or 'do nothing' cases?
Coprocessor	How do I hand work off to a specialised auxiliary unit (e.g. floating point)?

Exam tip: If asked to 'classify' a given instruction, first ask: does it touch memory? (Load/Store) → does it only compute on registers? (Arithmetic/Logical) → does it change the PC? (Jump/Branch). That three-question filter resolves 90% of classification questions instantly.

2. The Load-Store Philosophy

MIPS32 is a **load-store architecture**, meaning arithmetic and logic instructions are never allowed to touch memory directly – they only read and write processor registers. The *only* instructions permitted to access RAM are LOAD (bring data in) and STORE (push data out).

Analogy: Think of registers as a small workbench with a handful of trays right in front of you, and main memory as a huge warehouse shelf across the room. You are not allowed to hammer nails directly on the warehouse shelf – you must first carry the piece to your workbench (LOAD), work on it there, and carry the finished piece back to the shelf (STORE). This keeps every 'working' instruction fast and simple because it only ever deals with nearby, uniform-speed storage (registers).

Why it matters: Registers are physically tiny and sit right next to the ALU, so accessing them takes a fraction of a cycle. Memory is much slower and farther away. By forcing all computation to happen on registers, the CPU's critical arithmetic path never has to wait on slow memory – only the explicit LOAD/STORE instructions pay that cost, and only when the programmer actually needs it.

3. Memory Alignment: Why Addresses Must Be Multiples of 4

A word is 4 bytes. MIPS requires every word access to start at an address that is a multiple of 4 (i.e., the address's last two bits, in binary, are always 00).

Analogy: Imagine a bookshelf built with slots exactly one book wide. If every book is placed starting exactly at a slot boundary, you can grab any book in one motion. But if someone wedges a book so it spans two slots, grabbing it now takes two motions – reach into slot A, then also reach into slot B, and stitch the two halves together. That 'two motions' cost is precisely why misaligned words can need two memory cycles instead of one.

Why it matters: Memory hardware (RAM, caches) is physically organised in fixed-size rows. If the CPU always requests data aligned to those rows, one memory-bus transaction is guaranteed to contain the whole word. Enforcing alignment in the instruction set lets the hardware skip a lot of complicated boundary-checking logic, at the cost of requiring the compiler/programmer to lay data out carefully.

In the lecture's example, w1 begins at 0000H (a multiple of 4) so all four of its bytes sit inside one row → aligned. w2 begins at 0006H-ish (not a multiple of 4), so its bytes are split across two rows → misaligned. This is exactly the situation LWL/LWR/SWL/SWR exist to handle (see Section 4.4).

4. Load and Store Instructions – Deep Dive

Every load/store instruction is really answering four independent design questions at once. Once you see them as independent 'dimensions', the whole mnemonic zoo becomes predictable instead of arbitrary.

4.1 Dimension 1 – How many bytes?

The suffix letter tells you the operand size: **B**yte (1 byte), **H**alf-word (2 bytes), **W**ord (4 bytes, the 'default', no letter beyond LW/SW).

4.2 Dimension 2 – Signed or unsigned?

When you load a byte or half-word into a 32-bit register, the remaining upper bits must be filled in somehow. By default (no suffix), MIPS **sign-extends** – it copies the sign bit of the loaded value into all the extra upper bits, preserving the value's sign for two's-complement arithmetic. Appending **U** (e.g. LBU, LHU) instead **zero-extends** – it just pads with zeros, treating the value as a plain unsigned number. Note this distinction only matters for *loads narrower than a word*; stores simply write out the low-order bits, so there is no 'unsigned store'.

Why it matters: If a byte holding -1 (0xFF) is sign-extended, the 32-bit register becomes 0xFFFFFFFF, which is still -1 in two's complement – correct if you intended a signed char. If instead it is zero-extended, the register becomes 0x000000FF = 255 – correct if you intended an unsigned char. Same bits, opposite meaning: the instruction you pick encodes your intent about the data type.

4.3 Data sizes vs. operation support

Data Size	Load Signed	Load Unsigned	Store
Byte	YES	YES	YES
Half-word	YES	YES	YES
Word	YES	Only for MIPS64	YES
Unaligned word	YES	—	YES
Linked word (atomic modify)	YES	—	YES

Why 'unsigned word' is greyed out: A 32-bit word already fills every bit of a 32-bit register – there is no extra room to sign-extend or zero-extend into, so the signed/unsigned distinction is meaningless for a full-word load in 32-bit MIPS. It only becomes relevant again in MIPS64, where a 32-bit word is loaded into a 64-bit register and there IS extra room to fill.

4.4 Dimension 3 – Aligned vs. unaligned access

LWL (Load Word Left) and LWR (Load Word Right) work as a pair: LWL grabs the portion of the word from the current row (say the 'left' higher-address bytes) while LWR grabs the remaining portion from the next row, and together they assemble the full unaligned word. SWL/SWR do the reverse for stores. You almost never write these by hand – a compiler emits them automatically when it detects an unaligned access is unavoidable (e.g. packed structs).

4.5 Dimension 4 – Atomic read-modify-write

LL (Load Linked) and SC (Store Conditional) are used together to build synchronisation primitives such as locks or atomic counters on multi-core systems. LL loads a word and quietly 'flags' that address for monitoring; SC only actually writes back if no other processor touched that address in between – otherwise it fails and reports so, letting software retry. This is the foundation beneath higher-level constructs like mutexes.

4.6 Complete mnemonic reference

Type	Mnemonic	Function
Aligned	LB	Load Byte
Aligned	LBU	Load Byte Unsigned
Aligned	LH	Load Half-word
Aligned	LHU	Load Half-word Unsigned
Aligned	LW	Load Word
Aligned	SB	Store Byte
Aligned	SH	Store Half-word
Aligned	SW	Store Word
Unaligned	LWL	Load Word Left
Unaligned	LWR	Load Word Right
Unaligned	SWL	Store Word Left
Unaligned	SWR	Store Word Right
Atomic Update	LL	Load Linked Word
Atomic Update	SC	Store Conditional Word

Memory aid: Read any load/store mnemonic left to right as '[L/S] + [size: B/H/W] + [U if unsigned]'. LBU = Load + Byte + Unsigned. SWR is the odd one out (it belongs to the unaligned pair, 'R' = right half, not a data size).

5. Arithmetic and Logic Instructions – Deep Dive

Because MIPS is load-store, every ALU instruction's operands and results live entirely in registers. The five sub-categories differ only in *how many operands* and *where the second operand comes from*.

5.1 ALU Immediate – when one operand is baked into the instruction

These instructions take one register operand and one 16-bit constant encoded directly in the instruction word (no separate register needed). This is why they use the I-type encoding format (see Section 7).

Mnemonic	Function
ADDI	Add Immediate Word
ADDIU	Add Immediate Unsigned Word
ANDI	AND Immediate
LUI	Load Upper Immediate
ORI	OR Immediate
SLTI	Set on Less Than Immediate
SLTIU	Set on Less Than Immediate Unsigned
XORI	Exclusive-OR Immediate

Why LUI exists: A 16-bit immediate can only directly represent a fairly small constant. To build a full 32-bit constant, compilers first use LUI to place a 16-bit value into the *upper* half of a register (with the lower half zeroed), then ORI to fill in the *lower* 16 bits. This two-instruction idiom is the standard MIPS trick for loading arbitrary 32-bit constants.

5.2 ALU 3-operand – the 'normal' register arithmetic

Two source registers, one destination register – the workhorse R-type format.

Mnemonic	Function
ADD	Add Word
ADDU	Add Unsigned Word
AND	Logical AND
NOR	Logical NOR
SLT	Set on Less Than
SLTU	Set on Less Than Unsigned
SUB	Subtract Word
SUBU	Subtract Unsigned Word
XOR	Logical XOR

Why both ADD and ADDU exist: Both perform the identical bit-level addition – the only difference is what happens on overflow. ADD triggers an exception (trap) if the signed result overflows 32 bits, which is useful for catching programmer bugs in signed arithmetic. ADDU never traps, which is what you want for address arithmetic (pointers, array indices) where 'overflow' is often a meaningless or expected wraparound rather than an error.

5.3 ALU 2-operand and Shift instructions

Type	Mnemonic	Function
2-Operand	CLO	Count Leading Ones in Word
2-Operand	CLZ	Count Leading Zeros in Word
Shift	ROTR	Rotate Word Right
Shift	ROTRV	Rotate Word Right Variable
Shift	SLL	Shift Word Left Logical
Shift	SLLV	Shift Word Left Logical Variable
Shift	SRA	Shift Word Right Arithmetic
Shift	SRAV	Shift Word Right Arithmetic Variable
Shift	SRL	Shift Word Right Logical
Shift	SRLV	Shift Word Right Logical Variable

Logical vs. Arithmetic shift right – the key distinction: SRL (logical) always fills the vacated high bits with 0, treating the number as unsigned bits. SRA (arithmetic) fills the vacated high bits with a copy of the original sign bit, so a negative number shifted right stays negative – equivalent to dividing by a power of two while preserving sign. Shifting left has no such ambiguity, which is why there is only SLL, not an 'arithmetic left shift'.

Variable vs. fixed shift: Plain SLL/SRA/SRL/ROTR encode the shift amount as a constant 5-bit 'shamt' field baked into the instruction (0–31), decided at compile time. The 'V' (Variable) versions instead read the shift amount from a register at run time – needed whenever the shift amount is itself a computed value, e.g. shifting by a loop variable.

5.4 Multiply and Divide – why results need double width

Multiplying two 32-bit numbers can produce a result up to 64 bits wide (e.g. the largest 32-bit values multiplied together far exceed 32 bits). Division similarly produces two separate 32-bit results: a quotient and a remainder. Since an ordinary register is only 32 bits, MIPS provides a dedicated hidden register pair, **HI** and **LO**, purely to hold these double-width results.

Analogy: Think of HI:LO as a special two-drawer output tray attached to the multiplier/divider unit, separate from the normal 32 general-purpose registers. You use MFHI/MFLO ('move from') to copy a result out of that tray into a normal register you can keep working with, or MTHI/MTLO to load a value into the tray.

Mnemonic	Function
DIV	Divide Word
DIVU	Divide Unsigned Word
MADD	Multiply and Add Word
MADDU	Multiply and Add Word Unsigned
MFHI	Move from HI
MFLO	Move from LO
MSUB	Multiply and Subtract Word
MSUBU	Multiply and Subtract Word Unsigned
MTHI	Move to HI
MTLO	Move to LO
MUL	Multiply Word to Register

Mnemonic	Function
MULT	Multiply Word
MULTU	Multiply Unsigned Word

Result placement rules: for multiplication, LO gets the low 32 bits of the product, HI gets the high 32 bits. MADD/MSUB multiply and then **accumulate** into the existing HI:LO value (add or subtract) – useful for sums-of-products (dot products, digital filters) without an extra ADD instruction per term. For division, LO gets the quotient, HI gets the remainder.

The MUL exception: MUL breaks the HI/LO pattern on purpose – it writes only the low 32 bits of the product straight into an ordinary GPR, skipping HI/LO entirely. This is a convenience instruction for the extremely common case where the programmer already knows the product fits in 32 bits and doesn't want the extra MFLO step.

6. Jump and Branch Instructions – Deep Dive

All of these instructions answer the same question – 'what address does the Program Counter (PC) jump to next?' – but they differ in *how far* they can reach and *whether* they check a condition first.

6.1 PC-relative conditional branch

A signed 16-bit offset is added to the PC, but only if a condition is true (e.g. two registers are equal for BEQ). Because the offset is only 16 bits, branches can only reach a limited distance forward or backward from the current instruction – this is normally fine because most branches (loops, if-statements) jump only a short distance in the surrounding code.

6.2 PC-region unconditional jump

J and JAL use a 26-bit field (effectively 28 bits after the implicit $\times 4$ scaling, since instructions are word-aligned) to jump anywhere within the current 256 MB 'region' of memory. This is wide enough to reach most function calls within a normal-sized program.

6.3 Absolute (register) unconditional jump

JR and JALR take their target address directly from a register, so they can jump absolutely anywhere in the full address space – not limited to 256 MB. This is exactly what is needed to **return from a function**: the return address could be anywhere in memory, so it must be reachable via JR \$ra, not a fixed-size immediate offset.

Why it matters: This is the mechanism behind every function return in MIPS: JAL saves the return address into R31 (\$ra) when calling a function; when the function finishes, it executes JR \$ra to jump back to wherever it was called from – a different, unpredictable location each time, which is exactly why an absolute register jump (not a fixed offset) is required.

6.4 Link variants – saving the return address

Any instruction with 'L' for 'Link' in its name (JAL, JALR, BGEZAL) additionally copies the address of the *next* instruction into register R31 (\$ra) before jumping. This single register slot is how MIPS implements function calls without a hardware call stack – the compiler is responsible for pushing \$ra onto a software stack itself if a function needs to call another function (nested calls).

Type	Mnemonic	Function
Unconditional Jump (256 MB region)	J	Jump
Unconditional Jump (256 MB region)	JAL	Jump and Link
Unconditional Jump (256 MB region)	JALX	Jump and Link Exchange
Unconditional Jump (Absolute Address)	JALR	Jump and Link Register
Unconditional Jump (Absolute Address)	JALRHB	Jump and Link Register with Hazard Barrier
Unconditional Jump (Absolute Address)	JR	Jump Register
Unconditional Jump (Absolute Address)	JRHB	Jump Register with Hazard Barrier

Type	Mnemonic	Function
PC-Relative, compares two registers	BEQ	Branch on Equal
PC-Relative, compares two registers	BNE	Branch on Not Equal
PC-Relative, compares with zero	BGEZ	Branch on Greater Than or Equal to Zero
PC-Relative, compares with zero	BGEZAL	Branch on Greater Than or Equal to Zero and Link
PC-Relative, compares with zero	BGTZ	Branch on Greater Than Zero
PC-Relative, compares with zero	BLEZ	Branch on Less Than or Equal to Zero

Exam tip: If asked 'why can't a branch reach the whole 4GB address space, but a register jump can?' – the answer is purely about how many bits are available in the instruction to store the target: a branch's 16-bit offset is tiny compared to the 32 bits available in a full register.

7. Miscellaneous and Coprocessor Instructions

These instructions exist for control and bookkeeping purposes rather than data computation.

Type	Mnemonic	Function
System Call / Breakpoint	BREAK	Breakpoint
System Call / Breakpoint	SYSCALL	System Call
Trap (reg vs. reg)	TEQ / TGE / TGEU / TLT / TLTU / TNE	Trap if equal / $\geq$ / $\geq$ unsigned / $<$ / $<$ unsigned / $\neq$
Trap (reg vs. immediate)	TEQI / TGEI / TGEIU / TLTi / TLTiU / TNEI	Immediate versions of the above traps
Conditional Move	MOVf / MOVN / MOVt / MOVZ	Move on FP false / not-zero / FP true / zero
Prefetch	PREF	Prefetch Register+Offset
No-op	NOP	No Operation

Why 'trap' instructions exist alongside branches: A branch changes control flow smoothly as part of normal execution. A trap deliberately raises an *exception* when a condition holds – used for run-time checks like array bounds or divide-by-zero detection, where you want the OS/exception handler to intervene rather than silently continuing.

Why conditional MOVE instructions matter: MOVZ/MOVN let a compiler implement a simple if/else assignment (e.g. $x = \text{cond} ? a : b;$) *without* a branch at all – both possible values are computed unconditionally, and the conditional move just picks which one lands in the destination register. This avoids 'branch misprediction' penalties on the CPU's pipeline for very short, simple conditionals.

Coprocessor instructions (CP0–CP3)

- **CP0** – the System Control Coprocessor, built into the CPU chip; manages the virtual memory system and exception handling. This is the only coprocessor guaranteed to exist on every MIPS chip.
- **CP1** – reserved for the floating-point unit (FPU).
- **CP2** – available for implementation-specific extensions (vendor-defined).
- **CP3** – reserved for future extensions.

The idea of a coprocessor is to let the main CPU 'hand off' specialised work (like floating-point math) to a separate unit optimised for that job, instead of building every possible operation into the main ALU. These are not discussed in depth in this course.

8. Instruction Encoding – Building It From First Principles

Rather than memorising three tables, notice the encoding formats are simply *different ways to spend the same 32-bit budget*, depending on what an instruction needs to describe.

Why opcode is always bits 31–26: Every format keeps the 6-bit opcode in exactly the same position at the very top of the word. This means the CPU's decoder can look at the same 6 bits *before* knowing which format the rest of the instruction uses, and only then decide how to interpret the remaining 26 bits. Fixed field positions across formats is what makes hardware decoding fast and simple.

8.1 R-type (Register) – when 3 registers must be named

Used whenever an instruction needs up to three register operands (two sources, one destination) plus possibly a small constant shift amount. Since opcode alone can't distinguish all these operations (they all share opcode = 000000), a second field, **funct**, extends it.

Bits	31–26	25–21	20–16	15–11	10–6	5–0
Field	opcode	rs	rt	rd	shamt	funct
Meaning	6-bit opcode	Source reg 1	Source reg 2	Destination reg	Shift amount	Opcode extension

Worked example, step by step: encode `add $t1, $s1, $s2`

- Step 1 – identify each register's number: `$t1` = R9 (destination → rd), `$s1` = R17 (first source → rs), `$s2` = R18 (second source → rt).
- Step 2 – look up the fixed fields for ADD: opcode = 000000, funct = 100000 (shamt = 00000 since no shift is involved).
- Step 3 – convert each register number to 5-bit binary: R17 = 10001, R18 = 10010, R9 = 01001.
- Step 4 – assemble the fields in order: opcode, rs, rt, rd, shamt, funct.

opcode	rs (R17)	rt (R18)	rd (R9)	shamt	funct
000000	10001	10010	01001	00000	100000

8.2 I-type (Immediate) – when a constant is embedded

Used whenever an instruction carries a 16-bit constant instead of a third register: ALU-immediate instructions, all loads/stores (the constant is the offset added to a base register), and PC-relative branches (the constant is the branch offset).

Bits	31–26	25–21	20–16	15–0
Field	opcode	rs	rt	Immediate Data
Meaning	6-bit opcode	Source reg 1	Destination reg	16-bit immediate data

Worked example, step by step: encode `lw $t1, 48($s1)`

- Step 1 – identify registers: `$t1` = R9 (destination → rt for loads), `$s1` = R17 (base register → rs).
- Step 2 – opcode for LW = 100011.
- Step 3 – convert the offset 48 into 16-bit binary: 48 = 0000000000110000.
- Step 4 – assemble: opcode, rs, rt, immediate.

opcode	rs (R17)	rt (R9)	Immediate Data (48)
100011	10001	01001	0000000000110000

Note the naming flip: In load/store instructions, rt is the *destination* for loads but the *data source* for stores, while rs is always the base address register. This is a common point of confusion – always double check which

role *rt* plays depending on load vs. store.

8.3 J-type (Jump) – when only an address matters

Used by J and JAL, which need no register operands at all – just as much address space as possible. All remaining 26 bits after the opcode are devoted to the target address.

Bits	31–26	25–0
Field	opcode	Immediate Data (jump address)

Why the 26 bits become 28: Since every instruction is word-aligned (address's last 2 bits are always 00, see Section 3), those two low bits never need to be stored explicitly – the hardware simply assumes them to be 0 and appends them to the 26-bit field, effectively giving 28 bits of addressing range 'for free'.

8.4 Decode-time optimisation: prefetching before you know what you need

Here's a subtle but important hardware trick: some instructions need both *rs* and *rt* as inputs, others need only *rs* – but the CPU only discovers *which* once decoding is complete (it needs to have read the opcode/funct first). Rather than wait, the hardware speculatively reads *both* *rs* and *rt* from the register file in parallel with decoding, and simply discards whichever one turns out to be unused. Similarly, the 16-bit or 26-bit immediate field is eagerly sign-extended to 32 bits in case it's needed later in the pipeline. This overlapping of 'maybe-needed' work with decoding is a small but genuine example of instruction-level parallelism inside a single CPU core.

9. Addressing Modes – How an Operand's Location Is Computed

An 'addressing mode' answers: given the bits in the instruction, how do we compute the actual value or target address involved? Each mode trades off simplicity against reach.

Mode	Example	How the address/operand is formed	Typical use
Register	add \$s1, \$s2, \$s3	Operands are register contents directly – no address computation at all.	Ordinary computation
Immediate	addi \$s1, \$s2, 200	One operand is a constant embedded in the instruction itself.	Constants, small increments
Base	lw \$s1, 150(\$s2)	Register content (\$s2) + offset (150) = memory address.	Arrays, structs, stack variables
PC-relative	beq \$s1, \$s2, Label	PC + 16-bit signed offset = branch target address.	Short conditional branches
Pseudo-direct	j Label	(26-bit field × 4) combined with PC = jump target address.	Function calls, long jumps

Why 'base addressing' is the most-used mode in real programs: Reading array element $A[i]$ or a struct field almost always means 'take some base address, then add a computed offset'. Base addressing directly matches that pattern, which is why lw/sw with an offset is one of the most frequently executed instruction shapes in typical programs.

10. Worked Examples: Translating C to MIPS32

The general method for hand-translating C to MIPS: (1) assign a register to every variable that's already 'given' to you, (2) work out how many temporary registers you need for intermediate values, (3) translate operator by operator, left to right, reusing temporaries.

Example 1 – a single addition

C: $A = B + C$; ($B \rightarrow \$s2$, $C \rightarrow \$s3$, result $A \leftarrow \$s1$)

```
add $s1, $s2, $s3
```

Nothing more is needed – one C operator maps to exactly one ADD instruction.

Example 2 – chained expressions needing a temporary register

C: $A = B + C - D$; $E = F + A$; ($B \rightarrow \$s1$, $C \rightarrow \$s2$, $D \rightarrow \$s3$, $F \rightarrow \$s5$; $A \leftarrow \$s0$, $E \leftarrow \$s4$)

- The expression $B + C - D$ has two operators, so it cannot be done in one instruction – we need a **temporary register** (\$t0) to hold the intermediate result of $B+C$.
- First compute the addition into the temporary, then subtract D from that temporary into the final destination A .
- Only then can the second statement ($E = F + A$) run, since it depends on A 's finished value.

```
add $t0, $s1, $s2 # t0 = B + C
sub $s0, $t0, $s3 # A = t0 - D
add $s4, $s5, $s0 # E = F + A
```

Example 3 – array indexing (combining arithmetic with load/store)

C: $A[10] = X - A[12]$; ($\$s3$ = base address of array A , $\$s2 = X$, $\$t0$ = temporary)

- Each array element is a word (4 bytes), so element index i sits at byte offset $4 \times i$ from the base – this is why array indexing always involves a multiplication by the element size, done here at compile time since the indices (10, 12) are constants.
- $A[12]$ is therefore at address $\$s3 + 4 \times 12 = \$s3 + 48$; $A[10]$ is at $\$s3 + 4 \times 10 = \$s3 + 40$.
- First LOAD $A[12]$ into a temporary, subtract it from X , then STORE the result into $A[10]$.

```
lw $t0, 48($s3) # t0 = A[12]
sub $t0, $s2, $t0 # t0 = X - A[12]
sw $t0, 40($s3) # A[10] = t0
```

Example 4 – control constructs (if-statement)

C: if (x == y) z = x - y; (x→\$s0, y→\$s1, z←\$s3)

The key translation trick: MIPS branch instructions only jump when their condition holds; they have no 'else this whole block' construct. So an 'if (cond) { body }' is always translated by **inverting** the condition and branching *past* the body when the inverted condition is true – i.e. 'skip the body unless cond holds' becomes 'if NOT cond, jump past the body'.

```
bne $s0, $s1, Label # if x != y (i.e. NOT x==y), skip the subtraction
sub $s3, $s0, $s1 # z = x - y (only reached if x == y)
Label: ...
```

Exam tip: Whenever you see an 'if (A == B)' in C, expect a BNE in the assembly (not BEQ) – the comparison operator gets logically negated because the branch is used to *skip* the body, not to *enter* it. This inversion trips up many students in exams – practice it deliberately.

11. Self-Test: Can You Explain These Out Loud?

Real understanding means being able to explain the 'why', not just recite the 'what'. Try answering each of these without looking back at the notes:

- Why must every MIPS instruction be exactly 32 bits, and how does that speed up instruction fetch?
- Why is MIPS called a 'load-store' architecture, and what does that forbid?
- Why does a misaligned word sometimes need two memory cycles instead of one?
- Why do both ADD and ADDU exist when they compute the identical bits?
- Why does multiplication need a separate HI/LO register pair instead of just one destination register?
- Why can JR/JALR reach any address in memory, but BEQ/BNE cannot?
- Why is the opcode field always in the same bit position (31–26) across R, I, and J formats?
- Why does 'if (x == y)' typically compile into a BNE instruction rather than a BEQ?
- Why is base addressing (lw/sw with offset) the most common addressing mode in real code?

Study strategy: Cover the tables and try to redraw the R-type/I-type/J-type bit layouts from memory, then check yourself against Section 8. If you can regenerate the tables rather than just recognise them, you're ready for the exam.